GenAI Analysis Report

1. Tool and Access Methods:

a) GenAI Tool 1: ChatGPT (GPT-4o) accessed on web browser at chatgpt.com
b) GenAI Tool 2: Claude (Claude 3.5 Sonnet) accessed on web browser at claude.ai

2. Prompt Used:

"Generate C++ code using objects, not functions, to implement the following requirements:
• Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
  o Each digit from 1 to 9 must appear exactly once in each row.
  o Each digit from 1 to 9 must appear exactly once in each column.
  o Each digit from 1 to 9 must appear exactly once in each block.
• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• Implement a brute-force search using depth-first search with backtracking, in which we exhaustively try possibilities until we find a solution.
• Specifically: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. Identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values, return a no-solution-found result. Otherwise, try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e., a recursive call). If we get a no-solution-found to the first candidate, try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists. If a solution exists, we are guaranteed to find it by exhaustively trying all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five puzzle files.
• The output for each solution should include:
  o The puzzle file name (e.g., puzzle1.txt)
  o The puzzle from the file printed out.
  o The solution to the puzzle printed out or if no solution is found, "No solution found" printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out."

3. Raw Generated Code

NOTE: The code is split up into pieces through screenshots, as the code is too long for one photo.

ChatGPT:

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Sudoku {
private:
    int grid[9][9];
    int original[9][9];
    int solutionCount;

    // Check whether a number can be placed in a cell.
    bool isValid(int row, int col, int num) {
        // Check the row and column.
        for (int i = 0; i < 9; i++) {
            if (grid[row][i] == num)
                return false;

            if (grid[i][col] == num)
                return false;
        }

        // Find the top-left corner of the 3x3 block.
        int startRow = (row / 3) * 3;
        int startCol = (col / 3) * 3;

        // Check the 3x3 block.
        for (int i = startRow; i < startRow + 3; i++) {
            for (int j = startCol; j < startCol + 3; j++) {
                if (grid[i][j] == num)
                    return false;
            }
        }

        return true;
    }

    // Recursive depth-first search with backtracking.
    void solve() {
        int row = -1;
        int col = -1;
        bool emptyFound = false;

        // Find the first empty cell.
        for (int i = 0; i < 9 && !emptyFound; i++) {
            for (int j = 0; j < 9; j++) {
                if (grid[i][j] == 0) {
                    row = i;
                    col = j;
                    emptyFound = true;
                    break;
                }
            }
        }

        // If there are no empty cells, a solution is found.
        if (!emptyFound) {
            solutionCount++;

            cout << "\nSolution " << solutionCount << ":\n";
            printGrid(grid);
            return;
        }

        // Try every possible candidate from 1 to 9.
        for (int num = 1; num <= 9; num++) {
            if (isValid(row, col, num)) {
                // Place the candidate.
                grid[row][col] = num;

                // Recursively solve the remaining puzzle.
                solve();

                // Backtrack to try another candidate.
                grid[row][col] = 0;
            }
        }
    }
```

```
    // Print a Sudoku grid.
    void printGrid(int board[9][9]) {
        for (int i = 0; i < 9; i++) {
            if (i % 3 == 0 && i != 0)
                cout << "------+-------+------\n";

            for (int j = 0; j < 9; j++) {
                if (j % 3 == 0 && j != 0)
                    cout << "| ";

                if (board[i][j] == 0)
                    cout << "_ ";
                else
                    cout << board[i][j] << " ";
            }

            cout << '\n';
        }
    }

public:
    // Initialize the Sudoku object.
    Sudoku() {
        solutionCount = 0;

        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                grid[i][j] = 0;
                original[i][j] = 0;
            }
        }
    }

    // Read the puzzle from a file.
    bool loadPuzzle(string filename) {
        ifstream inputFile(filename);

        if (!inputFile) {
            cout << "Could not open " << filename << '\n';
            return false;
        }

        char ch;
        int count = 0;

        // Read digits and underscores, ignoring whitespace.
        while (inputFile >> ch && count < 81) {
            if (ch >= '1' && ch <= '9') {
                grid[count / 9][count % 9] = ch - '0';
                count++;
            }
            else if (ch == '_') {
                grid[count / 9][count % 9] = 0;
                count++;
            }
            // Other characters, such as grid separators, are ignored.
        }

        inputFile.close();

        if (count != 81) {
            cout << "Invalid puzzle format in " << filename << '\n';
            return false;
        }

        // Save the original puzzle for printing.
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                original[i][j] = grid[i][j];
            }
        }

        return true;
    }

    // Display the original puzzle and find every solution.
    void run(string filename) {
```

```
158         cout << "\n==================================\n";
159         cout << "File: " << filename << '\n';
160         cout << "Original Puzzle:\n";
161         printGrid(original);
162 
163         solutionCount = 0;
164 
165         solve();
166 
167         if (solutionCount == 0) {
168             cout << "\nNo solution found\n";
169         }
170         else {
171             cout << "\nTotal solutions found: "
172                  << solutionCount << '\n';
173         }
174     }
175 };
176 
177 int main() {
178     string filenames[5] = {
179         "puzzle1.txt",
180         "puzzle2.txt",
181         "puzzle3.txt",
182         "puzzle4.txt",
183         "puzzle5.txt"
184     };
185 
186     for (int i = 0; i < 5; i++) {
187         Sudoku puzzle;
188 
189         if (puzzle.loadPuzzle(filenames[i])) {
190             puzzle.run(filenames[i]);
191         }
192     }
193 
194     return 0;
195 }
```

Claude:

```cpp
// Object-oriented Sudoku solver: recursive depth-first search with backtracking.
// Finds ALL solutions of each puzzle file.
//
// Build:  g++ -std=c++17 -O2 -o sudoku sudoku.cpp
// Run:    ./sudoku                    (solves puzzle1.txt ... puzzle5.txt)
//         ./sudoku a.txt b.txt        (solves the files named on the command line)

#include <array>
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

// ---------------------------------------------------------------------------
// SudokuGrid: owns the 9x9 board, knows the rules, can load and print itself.
// A value of 0 means "empty".
// ---------------------------------------------------------------------------
class SudokuGrid {
public:
    static constexpr int SIZE = 9;
    static constexpr int BLOCK = 3;

    SudokuGrid() { for (auto& row : cells_) row.fill(0); }

    // Reads 81 cell characters from a file. Digits 1-9 are givens; '_' (also
    // '.' or '0') is a blank. Whitespace is ignored, so both "53__7____" and
    // "5 3 _ _ 7 _ _ _ _" layouts work. Returns false on I/O or format error.
    bool loadFromFile(const std::string& filename) {
        std::ifstream in(filename);
        if (!in) return false;

        int count = 0;
        char ch;
        while (in.get(ch)) {
            if (std::isspace(static_cast<unsigned char>(ch))) continue;
            if (count >= SIZE * SIZE) return false;  // too many cells
            int value;
            if (ch >= '1' && ch <= '9') value = ch - '0';
            else if (ch == '_' || ch == '.' || ch == '0') value = 0;
            else return false;                       // unexpected character
            cells_[count / SIZE][count % SIZE] = value;
            ++count;
        }
        return count == SIZE * SIZE;
    }

    int  get(int r, int c) const { return cells_[r][c]; }
    void set(int r, int c, int v) { cells_[r][c] = v; }

    // Finds the first empty cell (row-major order). False if the grid is full.
    bool findEmptyCell(int& row, int& col) const {
        for (int r = 0; r < SIZE; ++r)
            for (int c = 0; c < SIZE; ++c)
                if (cells_[r][c] == 0) { row = r; col = c; return true; }
        return false;
    }

    // Can value v go in (r,c) without violating row/column/block rules?
    // (The cell's own current content is ignored.)
    bool isValidPlacement(int r, int c, int v) const {
        for (int i = 0; i < SIZE; ++i) {
            if (i != c && cells_[r][i] == v) return false;  // row
            if (i != r && cells_[i][c] == v) return false;  // column
        }
        const int br = (r / BLOCK) * BLOCK, bc = (c / BLOCK) * BLOCK;
```

```
        for (int i = br; i < br + BLOCK; ++i)
            for (int j = bc; j < bc + BLOCK; ++j)
                if ((i != r || j != c) && cells_[i][j] == v) return false;  // block
        return true;
    }

    // List of digits that may legally be placed in cell (r,c).
    std::vector<int> candidates(int r, int c) const {
        std::vector<int> result;
        for (int v = 1; v <= 9; ++v)
            if (isValidPlacement(r, c, v)) result.push_back(v);
        return result;
    }

    // True if the filled-in cells don't already contradict each other.
    bool givensAreConsistent() const {
        for (int r = 0; r < SIZE; ++r)
            for (int c = 0; c < SIZE; ++c)
                if (cells_[r][c] != 0 && !isValidPlacement(r, c, cells_[r][c]))
                    return false;
        return true;
    }

    void print(std::ostream& os) const {
        for (int r = 0; r < SIZE; ++r) {
            if (r > 0 && r % BLOCK == 0) os << "------+-------+------\n";
            for (int c = 0; c < SIZE; ++c) {
                if (c > 0 && c % BLOCK == 0) os << "| ";
                os << (cells_[r][c] == 0 ? '_' : static_cast<char>('0' + cells_[r][c]));
                os << (c < SIZE - 1 ? " " : "");
            }
            os << '\n';
        }
    }

private:
    std::array<std::array<int, SIZE>, SIZE> cells_;
};

// ---------------------------------------------------------------------------
// SudokuSolver: exhaustive DFS with backtracking. Collects every solution.
// ---------------------------------------------------------------------------
class SudokuSolver {
public:
    explicit SudokuSolver(const SudokuGrid& puzzle) : working_(puzzle) {}

    // Returns all solutions found (empty vector => no solution exists).
    const std::vector<SudokuGrid>& solveAll() {
        solutions_.clear();
        if (working_.givensAreConsistent())
            search();
        return solutions_;
    }

private:
    // Recursive step: fill one more empty cell, or record a solution.
    void search() {
        int r, c;
        if (!working_.findEmptyCell(r, c)) {   // all 81 cells filled
            solutions_.push_back(working_);
            return;
        }
```

```
128         for (int v : working_.candidates(r, c)) {  // no candidates => loop is skipped
129             working_.set(r, c, v);                 // try it
130             search();                              // recurse with one more cell filled
131             working_.set(r, c, 0);                 // backtrack
132         }
133     }
134
135     SudokuGrid working_;
136     std::vector<SudokuGrid> solutions_;
137 };
138
139 // ----------------------------------------------------------------------------
140 // PuzzleRunner: loads one puzzle file, solves it, and reports the results.
141 // ----------------------------------------------------------------------------
142 class PuzzleRunner {
143 public:
144     explicit PuzzleRunner(std::string filename) : filename_(std::move(filename)) {}
145
146     void run(std::ostream& os) const {
147         os << "============================\n";
148         os << "Puzzle file: " << filename_ << "\n\n";
149
150         SudokuGrid puzzle;
151         if (!puzzle.loadFromFile(filename_)) {
152             os << "Error: could not read a valid 9x9 puzzle from this file.\n\n";
153             return;
154         }
155
156         os << "Puzzle:\n";
157         puzzle.print(os);
158         os << '\n';
159
160         SudokuSolver solver(puzzle);
161         const auto& solutions = solver.solveAll();
162
163         if (solutions.empty()) {
164             os << "No solution found\n\n";
165             return;
166         }
167
168         os << solutions.size() << (solutions.size() == 1 ? " solution" : " solutions")
169            << " found:\n\n";
170         for (std::size_t i = 0; i < solutions.size(); ++i) {
171             if (solutions.size() > 1) os << "Solution " << (i + 1) << ":\n";
172             solutions[i].print(os);
173             os << '\n';
174         }
175     }
176
177 private:
178     std::string filename_;
179 };
180
181 int main(int argc, char* argv[]) {
182     std::vector<std::string> files;
183     if (argc > 1) {
184         for (int i = 1; i < argc; ++i) files.emplace_back(argv[i]);
185     } else {
186         for (int i = 1; i <= 5; ++i)
187             files.push_back("puzzle" + std::to_string(i) + ".txt");
188     }
189
190     for (const auto& f : files)
191         PuzzleRunner(f).run(std::cout);
192
193     return 0;
194 }
```

4. Code Comparison:

In terms of correctness, both the code given by ChatGPT and Claude execute correctly when standard test inputs are used, successfully implementing a recursive depth first search with backtracking in C++ to solve partially completed 9x9 Sudoku grids. However, in terms of edge cases, ChatGPT's code contains an actual structural vulnerability inside its input parsing logic. Specifically, ChatGPT reads file inputs character-by character using standard stream extraction (inputFile >> ch), which can fail or misalign state if there is unexpected trailing whitespace or non-standard formatting across different operating systems. Additionally, ChatGPT manually copies the entire grid into a separate original trackng array and mixes printing logic directly inside the recursive solver function. Claude's program avoids these architectural problems by decoupling responsibilities, where it encapsulates the board state and file parsing safely inside its own dedicated SudokuGrid class, handles multiple input tokens (_, ., 0) cleanly, and validates initial board consistency (givensAreConsistent) before starting the search. Furthermore, Claude cleanly separates concerns by utilizing a SudokuSolver class to collect all valid solutions into a container and a PuzzleRunner class to manage file loading and formatted output streams.

As for execution time, both programs execute efficiently and have an identical worst-case time complexity of O(9^m) (where m represents the number of empty cells), as both of them exhaustively search the candidate possibilities across the 9x9 grid. Both programs complete the test suite in a fraction of a second. Claude is slightly better in runtime efficiency during grid printing and output display because it writes directly to standard streams with standard newline formating, avoiding the performance issues associated with unoptimized stream flushing.

In terms of space complexity, both programs maintain an O(m) stack space complexity regarding the maximum depth of the recursive call stack (at most 81 frames deep). Claude requires a small amount of additional heap memory to store the found solutions inside a std::vector<SudokuGrid>, while ChatGPT just stores a duplicate 9x9 array inside the class instance. Overall memory consumption across both implementations remains negligible and well within acceptable limits.

In terms of maintainability, Claude's implementation is significantly better structured and uses better standard C++ practices. Claude uses modern C++ features such as std::array, const-correctness for non-modifying accessor methods, encapsulated class separation, and clean dependency management. In contrast, ChatGPT relies on monolithic class design, traditional C-style raw arrays, and tighter coupling between algorithm execution and I/O display routines, making future maintenance and debugging much more complicated.

5. Baseline Selection and Improvements:

For the baseline selection, I chose the code given by Claude, since it is more maintainable, has a cleaner modular architecture, and uses a strong state-encapsulation strategy (SudokuGrid and SudokuSolver separation) that cleanly handles file parsing and board initialization.

As for the improvements, in terms of correctness, Claude's core logic was already fully functional and handled edge cases well, but I refined the file parsing and output handling to strictly match the required assignment specifications, ensuring all puzzle files (puzzle1.txt through puzzle5.txt) and their multiple

solutions are printed correctly alongside "No solution found" messages where applicable. For algorithm logic, the recursive depth first search with backtracking was already correct, so I kept the core solver architecture the same without any unnecessary modifications. As for execution time and space complexity, both stayed optimal at O(9^m) time complexity for the brute-force search space and O(m) space complexity for the recursive call stack and solution container, though I optimized memory usage slightly by passing grid and string parameters by const reference and utilizing reserve() where apropriate. Furthermore, the main improvements were made in maintainability, where I added a full prologue comment header and comprehensive in-line comments detailing block logic and authorship attributions (citing Claude as the source) while still preserving Claude's clean class definitions and modular structure.